

$$d \sim p(d(s_t, s_{t+1}) \mid M, D_{t,0 < t < k}) \quad (3)$$

Implementation The navigation map’s underlying graph is semi-Eulerian, alternating between horizontal and vertical edges, with 2^{k+1} possible spatial configurations for a k -hop navigation map. For each map and set of k navigation instructions, $k - 1$ question-and-answer (QA) instances, i.e. "what is the next step?" are created. Further implementation details are in Appendix A.1.

2.3 Visual Tiling

Introduced by [Gol66], polyomino tiling is a classic spatial reasoning challenge. We extend this concept to test the LLM’s ability to comprehend, organize, and reason with shapes in a confined area, thus enhancing the evaluation of spatial reasoning skills. As depicted in Figure 3, the task involves a rectangle with unfilled cells and various polyomino pieces, like the I-tetromino made of four aligned squares. The model must select the appropriate polyomino variant, such as choosing the orientation for the I-tetromino, to solve the QA puzzle. Task instructions are provided in Figure 7 in appendix.

Formulation The model is presented with a rectangle R masked with k unique polyominoes $MP = \{mp_1, \dots, mp_k\}$, 2 corresponding variants of each polyomino $v_{i \leq k} = \{v_{i1}, v_{i2}\}$, and a polyomino query $q \in MP$. Visual tiling task is to identify the correct variant of q , as defined in Equation 4.

$$v \sim p(v_q \mid R, \{mp_1, \dots, mp_k\}, \{v_{11}, v_{12} \dots, v_{k1}, v_{k2}\}, q) \quad (4)$$

Implementation The dataset comprises valid spatial arrangements generated through existing algorithms[ES03, GN07], with random masking of polyominoes to create QA puzzles. Details are provided in Appendix A.2.

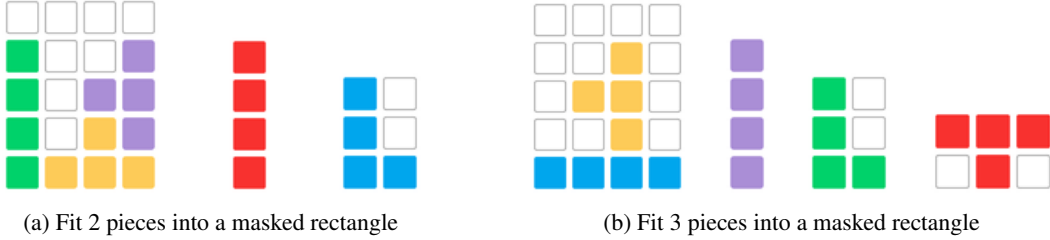


Figure 3: Example of visual tiling with masked polyomino pieces. Variants of those polyomino pieces including rotation and reflection are not shown in this figure.

3 Visualization-of-Thought Prompting

Considering the way humans process spatial information during tasks like navigation, it’s common to create mental images, such as maps, to enhance spatial awareness or simulating movements to inform decision-making. Our objective is to elicit the spatial awareness of LLMs and ground their reasoning by visualizing the consequence of their intermediate reasoning steps.

We introduce **Visualization-of-Thought (VoT)** prompting: **"Visualize the state after each reasoning step."** This new paradigm for spatial reasoning aims to generate reasoning traces and visualizations in an interleaved manner. Qualitative results of this approach are presented in Figure 4.

We use p_θ to denote a pre-trained LM with parameters θ , x, y, z to denote a language sequence, and v to denote a visualization sequence in text form. In a multi-hop spatial reasoning task with input x , CoT prompting generates a series of intermediate steps $z_1, \dots, z_n$, each step $z_i \sim p_\theta(z_i \mid x, z_{1 \dots i-1})$ is sampled sequentially, followed by the output $y \sim p_\theta(y \mid x, z_{1 \dots n})$. As shown in Figure 1, **VoT** prompting enhances this process by adding a visuospatial sketchpad to each intermediate step z_i , then the subsequent step z_{i+1} is sampled conditioned on prior steps $z_{1 \dots i}$ and visualizations $v_{1 \dots i}$.

As defined in the Equation 5 and 6, it forms interleaved reasoning traces and visualizations. A qualitative comparison between outputs of VoT and CoT is provided in Figure 8a in appendix.

$$v_i \sim p_\theta(v_i \mid \text{prompt}_{\text{VoT}}, x, z_{1 \dots i}, v_{1 \dots i-1}) \quad (5)$$

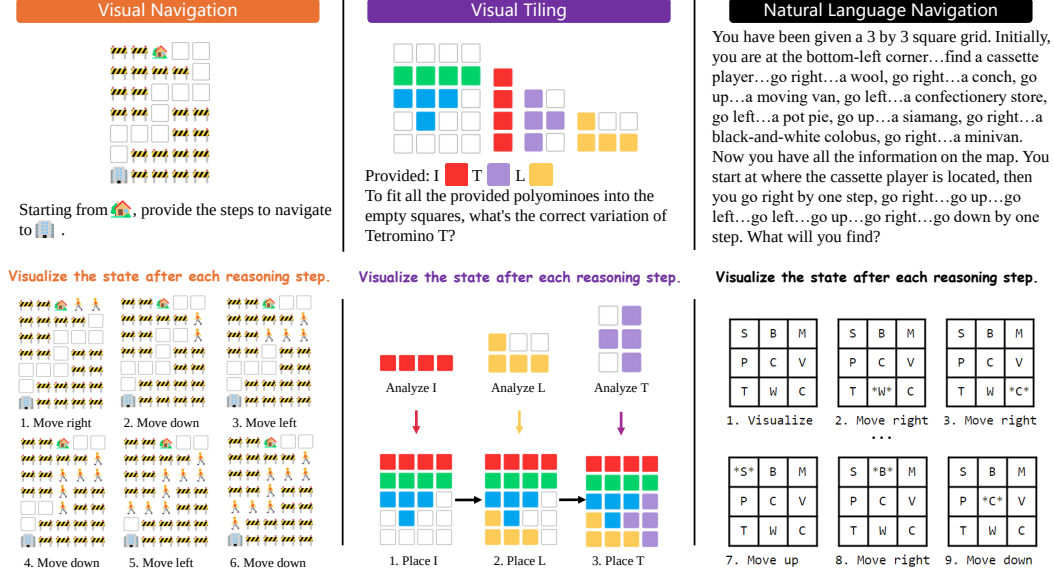


Figure 4: Examples of VoT prompting in three tasks, where LLM generates 2D grids as text-form *mental images*. The generated reasoning traces and visualizations form an interleaved sequence to track the state over time. The 2D grids in the input and responses are composed of special characters. Full responses could be found in Appendix B.

$$z_{i+1} \sim p_{\theta}(z_{i+1} \mid \text{prompt}_{VoT}, x, z_{1\dots i}, v_{1\dots i}) \quad (6)$$

This reasoning paradigm enables LLMs with visual state tracking. We introduce the concept of a **state**, denoted as $s_i = [x, z_{1\dots i}, v_{1\dots i-1}]$ representing a partial solution at step i with the input, the sequence of intermediate steps $z_{1\dots i}$ and the sequence of visualizations $v_{1\dots i-1}$.

$$\begin{aligned} v_i &\sim p_{\theta}(v_i \mid \text{prompt}_{VoT}, x, z_{1\dots i}, v_{1\dots i-1}) \\ &\sim p_{\theta}(v_i \mid \text{prompt}_{VoT}, s_i) \end{aligned} \quad (7)$$

As shown in Equation 7, visual state tracking is implemented by generating the visualization v_i as representation of the internal state s_i after each reasoning step z_i (e.g. v_i could be a grid of the navigation map marked with path or a filled rectangle). Grounded by the visual state tracking sequence, the subsequent state is derived by $s_{i+1} \sim p_{\theta}(s_{i+1} \mid \text{prompt}_{VoT}, x, s_i, v_i)$. This mechanism allows for the derivation of subsequent states, reflecting spatiotemporal causality and enhancing the spatial reasoning capabilities of LLMs in a grounded context.

4 Experiment

4.1 Setup

For the visual tasks where a counterpart image exists for each text input, we conduct additional experiments with a multimodal model. Specifically, we adopt GPT-4 [OA⁺23] and GPT-4 Vision [Ope23] via Azure OpenAI API as they're state of the art LLM and multimodal model respectively. API settings are *temperature* 0 as greedy decoding and *top p* 1, with model versions of 1106-preview and vision-preview. For all experiments we adopt **zero-shot** prompting.

Depending on whether the LLM is explicitly prompted to visualize intermediate steps, we experiment with three settings of GPT-4, including zero-shot CoT prompting(**GPT-4 CoT**), **GPT-4 w/o Viz** where visualization is explicitly disabled during reasoning, and VoT prompting (**GPT-4 VoT**). Additional setting of GPT-4 Vision with counterpart image input is **GPT-4V CoT**. Prompts are as following:

- **GPT-4 CoT**: Let’s think step by step.
- **GPT-4 w/o Viz**: Don’t use visualization. Let’s think step by step.
- **GPT-4V CoT**: Let’s think step by step.
- **GPT-4 VoT**: Visualize the state after each reasoning step.

Task instructions and examples could be found in Appendix B.

4.2 Dataset

Natural Language Navigation We generate 200 square maps of size 3x3 which is described by 9 landmarks in snake order traversal, and a set of navigation instructions.

Visual Navigation We generate 496 navigation maps and 2520 QA instances in total, covering various map sizes, up to 7x9 and 9x7. The data distribution is provided in Table 4 in appendix.

Visual Tiling We first generate multiple unique configurations to fill a 5 x 4 rectangle with 5 polyomino pieces including two I tetrominoes, two T tetrominoes and one L tetromino. Then we randomly masked two or three pieces of different types and generate QA instance for each masked pieces. The total number of QA instances is 796, and we show dataset details in Table 5 in appendix.

4.3 Metric

We extract the answer from model output by pattern matching. For tasks except for route planning, we calculate accuracy by Equation 8. We adopted sub-string matching[†] as $f_{correct}$ to determine correctness.

$$acc = \sum_i^n f_{correct}(extracted_answer, ground_truth)/n \quad (8)$$

For the route planning task which predicts a sequence of navigation instructions, we reject any sequences exceeding 100 instructions, considering them to be random guesses. We then normalize the navigation instructions by executing each navigation instruction. Those instructions which violate navigation rules will be ignored. The length t of normalized instruction sequence is considered as the temporal distance against the starting point. Given the ground-truth of k navigation instructions, the completing rate of route planning is t/k . For the dataset of n maps, we report two metrics including:

1. Average completing rate: $\sum_i^n t_i/k_i/n$. Average completing rate among all instruction sequences, reflecting LLM’s effectiveness of route planning.
2. Success rate: $\sum_i^n (t_i == k_i)/n$. This metric represents the proportion of instruction sequences with $t = k$, i.e., reaching the destination.

4.4 Results

As illustrated in Table 1, **GPT-4 VoT** significantly outperforms other settings in all tasks across all metrics. The significant gap when comparing GPT-4 VoT with GPT-4V CoT and GPT-4 w/o Viz demonstrates that effectiveness of visual state tracking, which allows LLMs visually interpret their actions within an grounded world. And in the natural language navigation task, **GPT-4 VoT** outperforms **GPT-4 w/o Viz** by 23.5%. In the visual tasks, the noticeable performance gap between **GPT-4 CoT** and **GPT-4V CoT** indicates that LLM grounded with 2D grid could possibly outperform a MLLM in challenging spatial reasoning tasks.

On the other hand, performance of GPT-4 VoT is still far from perfect in all tasks, especially in the most challenging route planning task. Despite these tasks are relatively easy for humans, performance of LLMs drops significantly as task difficulty increases. Details on performance trends across difficulty levels are provided in figure 9 and table 6 in appendix.

[†]We use this term for simplicity. In natural language navigation tasks, LLMs often output additional words in the extracted answer besides the expected object name. For example, "Answer: You will find ...". In this case, sub-string matching is adopted without affecting the correctness. Otherwise, exact matching is adopted for multiple choice questions in visual tasks.